Vivek Yadav (RA2311003011896)

EXPERIMENT: Ansible on AWS – Deploy Nginx

AIM

To launch EC2 instances on AWS, configure a control node and managed nodes, and use **Ansible** to automate the deployment of an Nginx web server on multiple instances.

ALGORITHM / PROCEDURE

1. **Launch EC2 Instances**
   - Three EC2 instances were created using Ubuntu 24.04.
   - One instance was designated as the **control node** and the other two as **managed nodes**.
   - A common key pair was used for all instances.
2. **Configure Security Group**
   - Allowed SSH (port 22) for remote access.
   - Allowed HTTP (port 80) for web access.
   - Enabled internal communication between instances.
3. **Connect to Control Node**
   - SSH connection was established from the local machine to the control node via EC2 Instance Connect.
4. **Install Ansible**
   - Ansible was installed on the control node using:

```
sudo apt update -y
sudo apt install ansible -y
```

5. **Copy SSH Key**
   - The `.pem` key file was copied to the control node and permissions were set.
6. **Create Ansible Inventory**
   - A file (`inventory.ini`) was created listing the private IP addresses of the managed nodes.
7. **Test Connectivity**
   - Used Ansible ping module:

```
ansible all -i ~/ansible-lab/inventory.ini -m ping
```

8. **Create Playbook**
   - A playbook (`deploy.yml`) was written to:

- Update packages
- Install Nginx
- Create a custom web page
- Start and enable the service

9. Execute Playbook

- Ran:
- Tasks were successfully executed on both managed nodes.

```
ansible-playbook -i inventory.ini deploy.yml
```

10. Verification

- Accessed managed node public IPs in a browser.
- Verified Nginx deployment with a custom webpage.

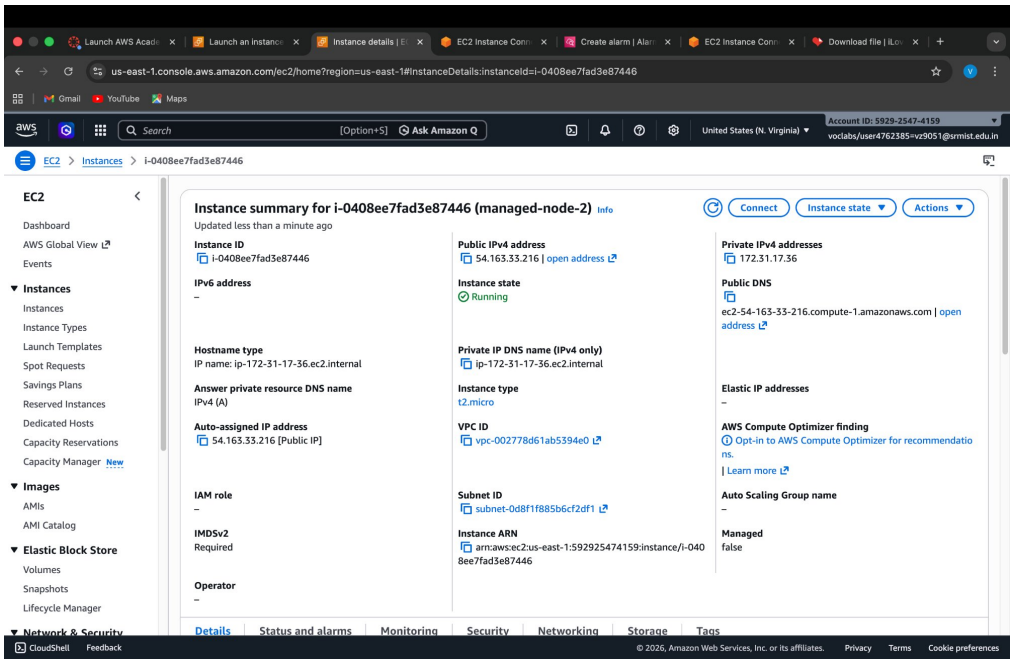

Figure 1: EC2 Instance Details – Managed Node

Description:

This figure shows the AWS EC2 Instance summary for managed-node-2 (i-0408ee7fad3e87446). The instance is in the Running state with a Public IPv4 address of 54.163.33.216 and Private IPv4 address 172.31.17.36, configured within the us-east-1 region. The instance type is t2.micro running Ubuntu 24.04.

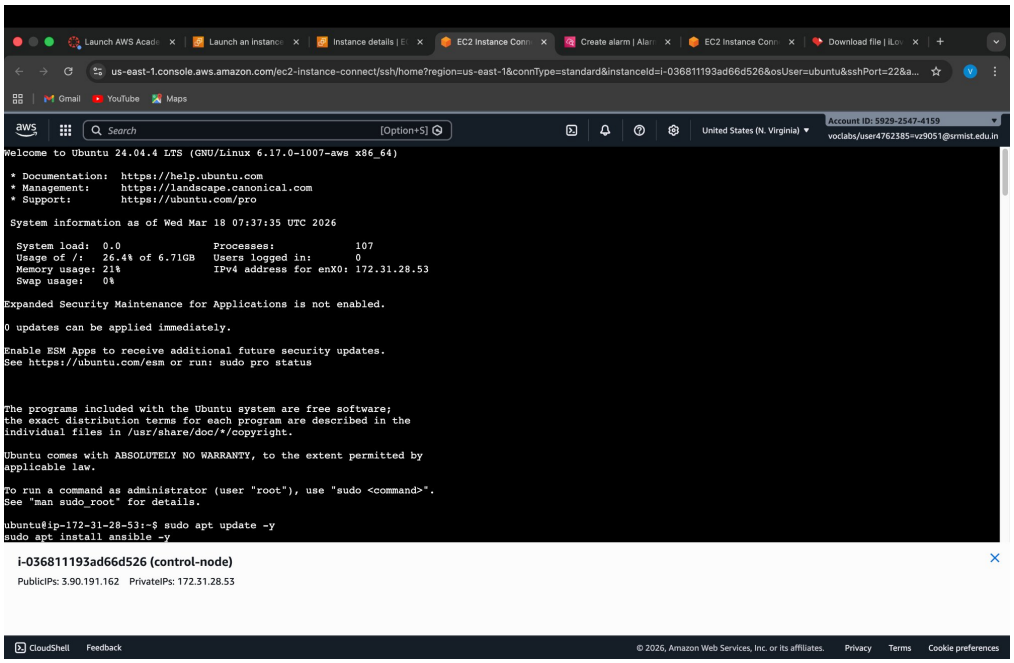

Figure 2: SSH Login to Control Node

Description:

This figure shows the successful SSH connection to the control node instance using EC2 Instance Connect. The Ubuntu 24.04.4 LTS terminal is displayed with system information as of March 2026. The control node (ip-172-31-28-53) is ready, and the commands `sudo apt update -y` and `sudo apt install ansible -y` are being executed to prepare the environment.

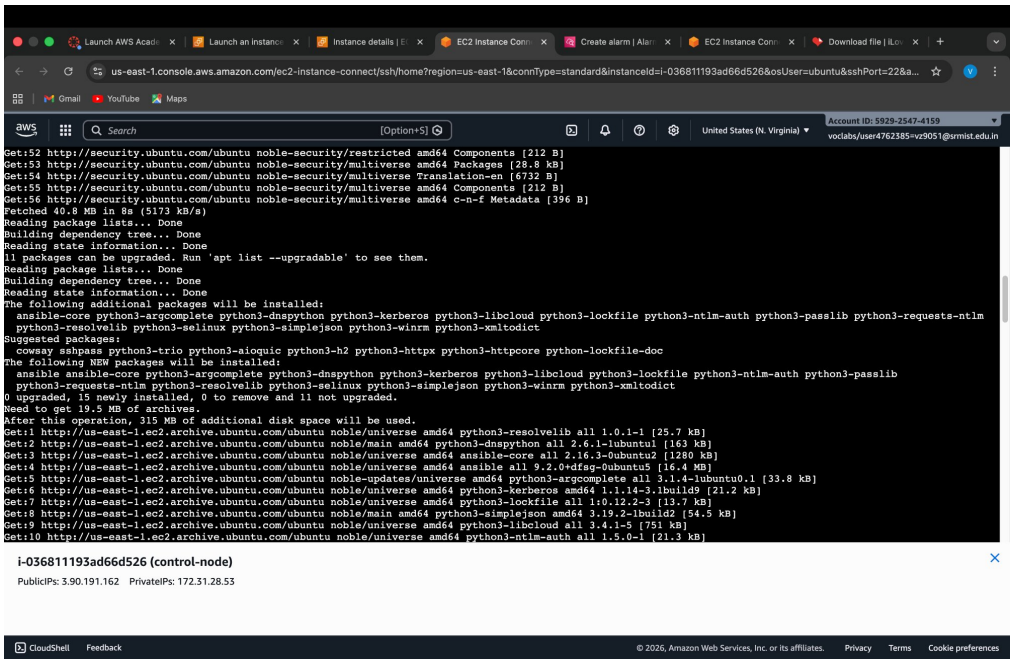

Figure 3: Updating Package Cache on Control Node

Description:

This figure shows the output of `sudo apt update` running on the control node. The system fetches package lists from Ubuntu noble (24.04) repositories including main, universe, restricted, and multiverse components, as well as security updates. The package database is successfully refreshed, indicating 11 packages can be upgraded.

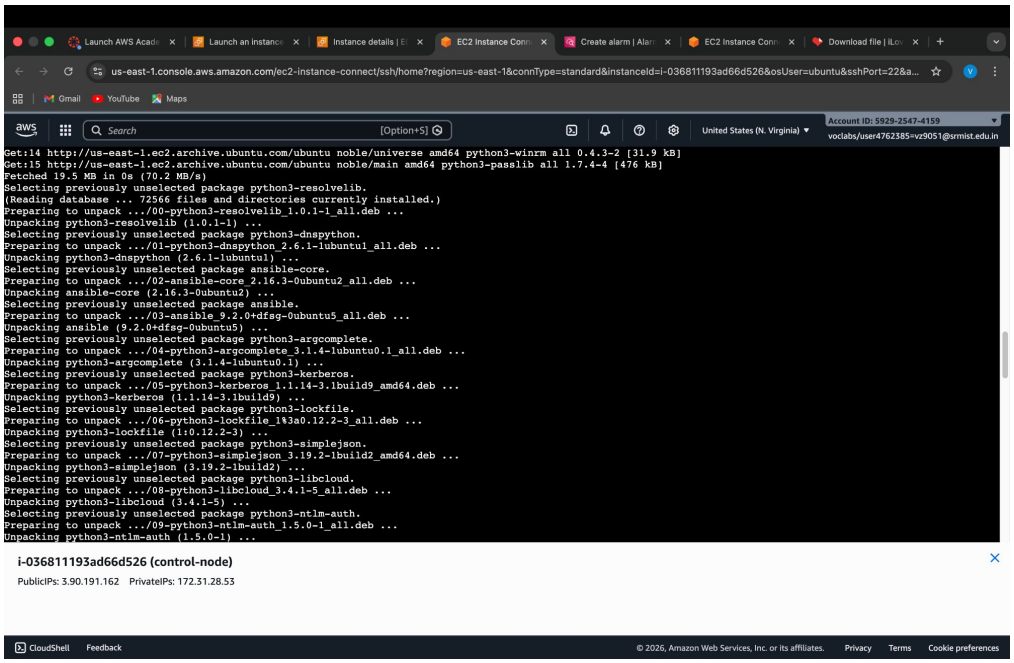

Figure 4: Installation of Ansible on Control Node

Description:

This figure shows Ansible and its dependencies being downloaded and installed on the control node. Packages including ansible-core, python3-argcomplete, python3-kerberos, python3-lockfile, python3-ntlm-auth, and others are fetched from Ubuntu Noble repositories. The installation requires approximately 315 MB of additional disk space.

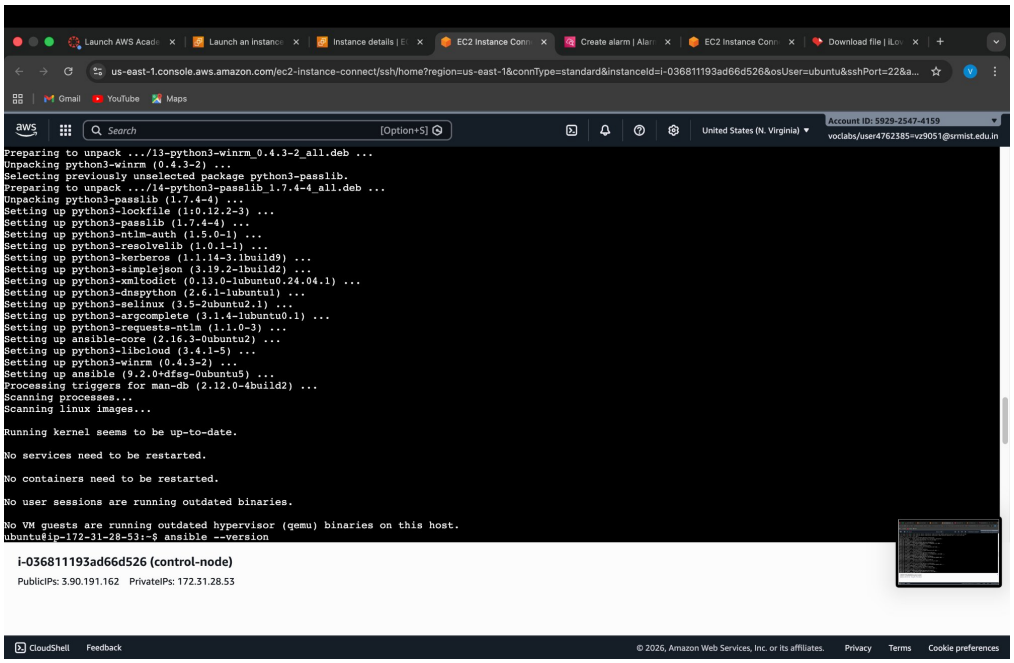

Figure 5: Ansible Installation Complete

Description:

This figure shows the completion of the Ansible installation process on the control node. All packages including ansible (9.2.0) and ansible-core (2.16.3) are successfully set up and configured. The system scanning confirms no services need to be restarted after installation, and the terminal is ready for the next command.

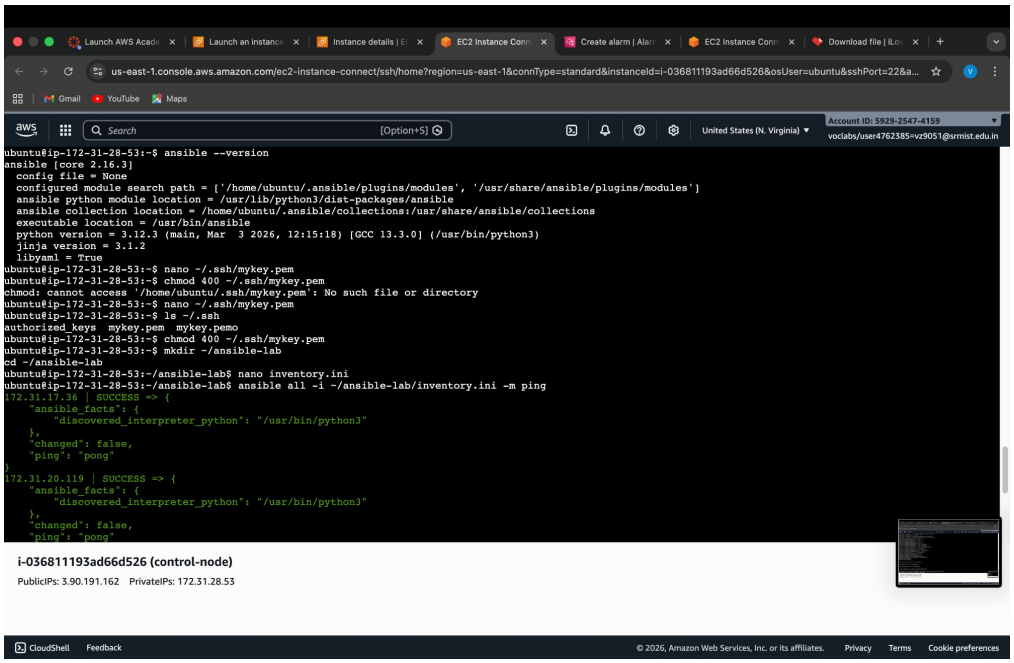

Figure 6: Ansible Version Verification and Connectivity Test

Description:

This figure shows the verification of the Ansible installation using `ansible --version`, confirming Ansible core 2.16.3 with Python 3.12.3. It also shows the SSH key setup (`chmod 400 ~/.ssh/mykey.pem`), creation of the `~/ansible-lab` directory, inventory file configuration, and successful ping test showing SUCCESS/pong responses from both managed nodes 172.31.17.36 and 172.31.20.119.

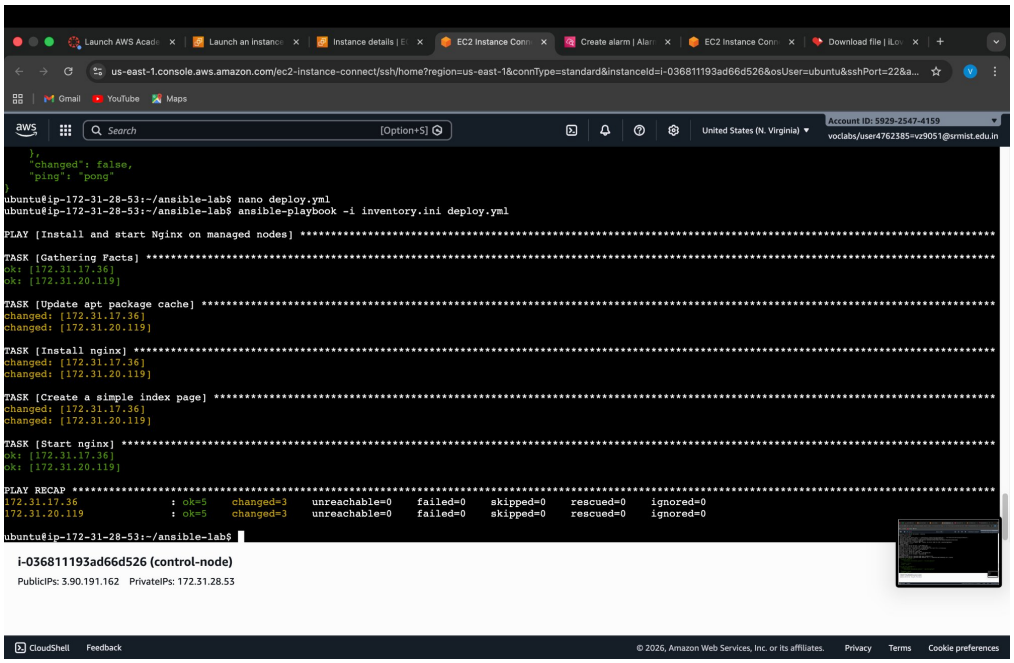

Figure 7: Ansible Playbook Execution on Managed Nodes

Description:

This figure shows the execution of the Ansible playbook (`ansible-playbook -i inventory.ini deploy.yml`) from the control node. The playbook performs tasks including Gathering Facts, updating the apt package cache, installing Nginx, creating a simple index page, and starting Nginx. The PLAY RECAP confirms ok=5, changed=3 for both nodes 172.31.17.36 and 172.31.20.119 with zero failures or unreachable hosts.

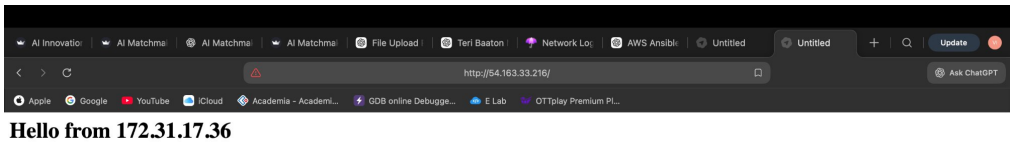

Figure 8: Verification of Nginx Deployment – Managed Node 1

Description:

This figure shows the successful verification of the Nginx web server deployment on managed node 1. The browser accesses the public IP address 54.163.33.216 and displays the custom page **"Hello from 172.31.17.36"**, confirming that the Ansible playbook correctly installed and configured Nginx on the first managed node.

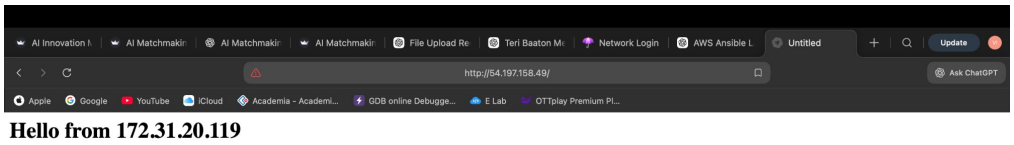

Figure 9: Verification of Nginx Deployment – Managed Node 2

Description:

This figure shows the successful verification of the Nginx web server deployment on managed node 2. The browser accesses the public IP address 54.197.158.49 and displays the custom page **"Hello from 172.31.20.119"**, confirming that the Ansible playbook correctly installed and configured Nginx on the second managed node as well.

RESULT

The experiment was successfully completed. Ansible was used to automate the installation and configuration of the Nginx web server on multiple AWS EC2 instances. The web server was successfully deployed and verified through a web browser.